[redacted]

Advanced Scripting Project

GitHub Link:
https://github.com/Advanced-Scripting-2026/asc2026-project-assessment-MaelConejos

Maël Conejos Crusson
17/04/2026

Table of Contents

Chapter 1: Introduction 2
Chapter 2: Methodology 2
Function 1: detectWebNotFoundAccess 2
Function 2: detectWebForbiddenAccess 3
Function 3: detectSSHFailedAuth 3
Function 4: detectAdminFailedAuth 4
Function 5: detectPSExecPatterns 4
Function 6: detectPSDataExfiltration 4
Function 7: detectIOCMatches 4
Chapter 3: Example Results 5
Chapter 4: Conclusion 6

Chapter 1: Introduction

The purpose of this PowerShell tool is to detect suspicious activities by scanning log files, such as web server access files, SSH login files, and PowerShell log files.

The tool I created performs 9 total detections across 3 files: access.log, auth.log, and powershell_log.txt:

- For the web server log file (access.log), it detects repeated 404 'Not Found' and 403 'Forbidden' errors.
- For the SSH log file (auth.log), it detects repeated failed login attempts and admin/root failed login attempts.
- For the PowerShell log file, it detects suspicious patterns used in commands as well as detecting potential data exfiltration.
- And finally, this tool uses the ioc.txt file which contains a list of malicious IPs and web paths, and checks for matches in the 3 log files.

Once all files have been scanned and suspicious activities detected, a report is generated to provide an overview of the scans.

Chapter 2: Methodology

The entire system is divided into 4 parts, the configuration, functions, report, and main:

- The first part is the 'configuration.ps1' file in the scripts folder. It contains all essential variables for the project: log & report file locations, alert counts (set to 0 by default), hits (set to 0 by default), and the thresholds.
- The second part is the 'detection_functions.ps1' file in the scripts folder. The file contains the detection functions of which there are 7 in total. They are similar to each other but overall serve different purposes.
- The third part is the 'report_generator.ps1' file in the scripts folder. It has a function creating, displaying, and saving the report.
- The final part is the 'run.ps1' script in the project folder. It runs all the functions including the report generator.

Most detection logic is similar across the functions: the functions use a log file as parameter and then search through that log file for a specific string and check whether the hits exceed the threshold. If they do, then the alert status is set to 1 and the hits count is recorded.

Function 1: detectWebNotFoundAccess

This function begins by using the web server log file as a parameter.

It splits the file path to get only the file name to display it with 'Write-Host' so that the function lets the user know what file it is currently working on.

It then assigns a variable ('$notFoundRequests') all lines of the log file that contain the string ' 404 ' using 'Select-String'.

Then, the function checks with an 'if' statement if the number of times the string has appeared (using '.Count') is equal to or exceeds the threshold set in 'configuration.ps1' ('$webNotFoundThreshold').

If the count is equal or exceeds the threshold, then the alert variable associated ('$webNotFoundAlert') gets a +1, the hits variable associated ('$webNotFoundHits') will get the hits count, the user will receive a message indicating an alert with the number of hits and the first 5 lines found with that string, using 'foreach' to iterate through the '$notFoundRequests' and 'Select-Object -First 5' to only pick the first 5 of the log file.

If no 404 errors are found, the user will then receive a message indicating no errors were found.

Checking for 'Not Found' errors in a web server log file is important because if there are an unusual amount of them, such as 5 or over (which is what I set my threshold to), then this could indicate an attacker attempting to map a site structure or trying to access specific parts of a website, like Reconnaissance.

Function 2: detectWebForbiddenAccess

This function operates exactly as the first one but replaces the 404 error by ' 403 '.

Forbidden errors are important to check in a log file because they indicate that a user has tried to access restricted content and the server has blocked the request. This could indicate an attacker trying to access information, like Escalation of Privilege. I set the threshold to 1 since any 403 error is suspicious.

Function 3: detectSSHFailedAuth

This function also works the same as the previous two, but this time it checks for a 'Failed Password' string in the SSH log file instead to detect repeated failed passwords.

Checking for an unusual number of failed passwords can help determine whether an attacker is attempting to brute-force a SSH connection or not. I set the threshold for failed passwords to 3 which I think is the normal amount for someone to fail their password, anywhere above that and it becomes suspicious.

Function 4: detectAdminFailedAuth

This function is the same as the last, but with the addition of '*admin' and '*root' to the strings 'Failed password' to specifically identify people attempting to log in as admin or root.

If this failed authentication is detected even once (I set my threshold to 1), the user is alerted. Considering admin is the user that has all permissions within a system, any failed authentication attempt can be considered suspicious.

Function 5: detectPSExecPatterns

This function now concerns the PowerShell log file. This function has the same logic as the previous, but searches for several risky commands in the log file: '-EncodedCommand', '-ExecutionPolicy Bypass', 'Invoke-WebRequest', and 'DownloadString'.

The presence of these strings in the log file may indicate suspicious execution patterns, such as hiding malicious scripts through encoding using '-EncodedCommand', or bypassing PowerShell's security policies using '-ExecutionPolicy Bypass' or downloading malicious payloads from the internet using 'Invoke-WebRequest' and 'DownloadString'. I set the threshold to 1 since the presence of any of these can reasonably be cause for concern.

Function 6: detectPSDataExfiltration

This function also uses the PowerShell log file. Unlike the last one, this one searches for two strings separately, and only the presence of both strings triggers the alarm. The strings in question are 'Compress-Archive' and 'Invoke-WebRequest -Upload'.

When both of these strings are found in a log file, it may indicate data exfiltration, such as archive creation ('Compress-Archive') right before network upload ('Invoke-WebRequest -Upload'). The threshold is only set to 1 as if both are present in a log file, it is suspicious.

Function 7: detectIOCMatches

This function differs from the rest. It firstly takes in two parameters: the Indicators of Compromise file ('$iocFile' or 'ioc.txt') containing known malicious IPs and web paths, and the log file ('$logFile', which may be any of the three mentioned log files).

It then creates a variable ('$iocFileContent') that extracts everything from inside the IoC file. It iterates through this text line by line (using 'foreach ($line in $iocFileContent)') and checks only for lines that are not empty (using 'if ($line -ne "")' in case the IoC file contains empty lines), and adds to the variable '&suspiciousConnections' all lines in the log file which contain a malicious IP or web path.

Again, the threshold is only set to 1 as the presence of any known malicious IPs or web paths is an immediate alert.

Chapter 3: Example Results

The tool is very simple to use, all that is needed to be done is to run the main 'run.ps1' script on PowerShell with the command '.\run.ps1' or '.<path to file>\run.ps1'. And it can be run from anywhere without the user needing to be in the project folder, as the four screenshots below indicate along with the output (Fig. 1-4).

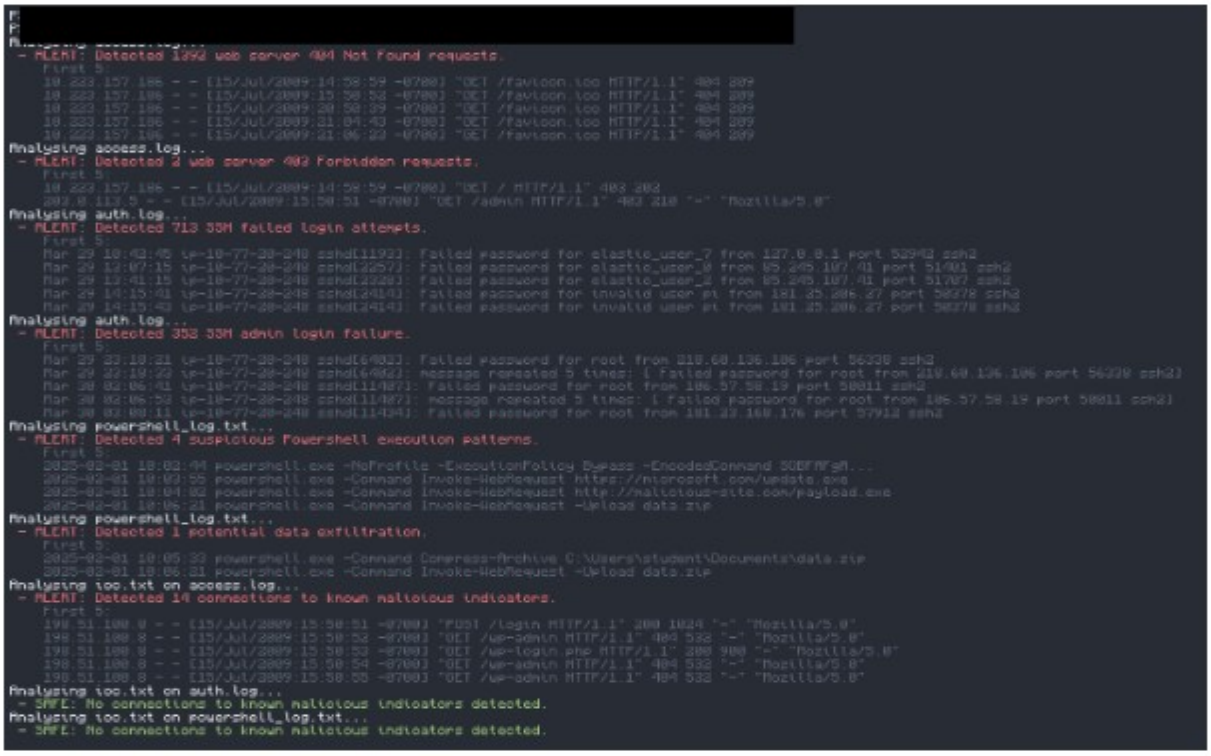


Figure 1: Running the file from the project folder (1)

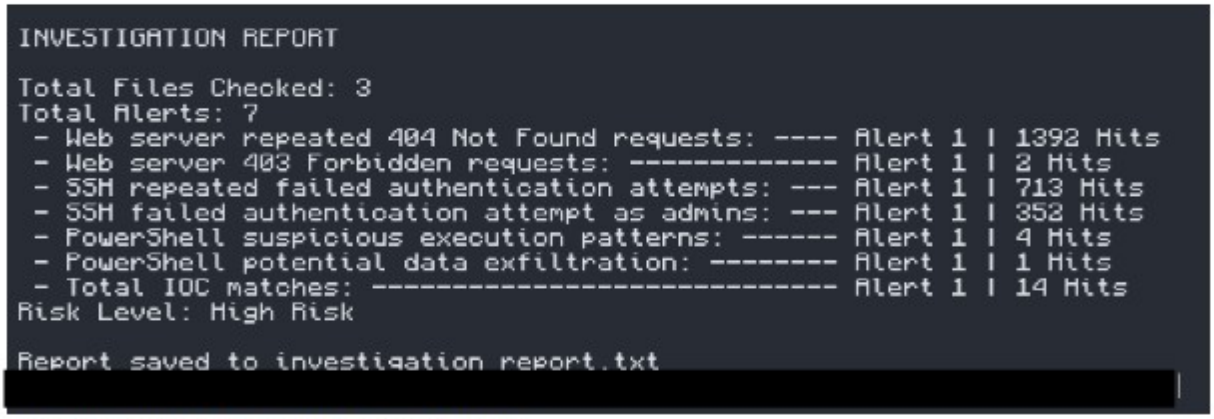


Figure 2: Running the file from the project folder (2)

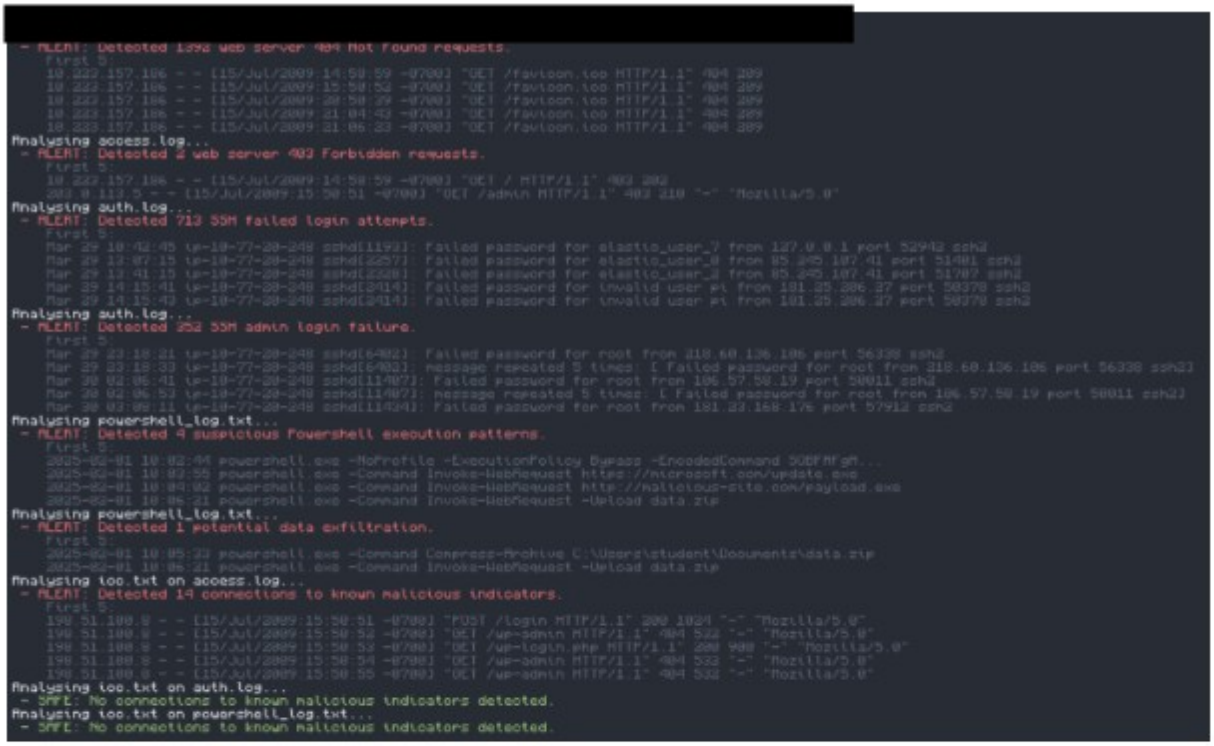


Figure 3: Running the file from a home folder and using an absolute path (1)



Figure 4: Running the file from a home folder and using an absolute path (2)

Chapter 4: Conclusion

This tool is a good tool for detecting suspicious activity in log files, but it does have its limitations.

Its strengths are that the functions aren't hardcoded, meaning that you can run any log file in any of the functions and it would work. Another strength is that all separated parts are assembled into a single file, meaning only one command is needed to be executed to run everything. And its final strength is that the code is divided, meaning everything is more or less modular and reusable, this makes it easy to segment, edit, swap, and change files.

A limitation however is that if you were to add any new log file to detect, you would have to manually add it as a variable in the 'configuration.ps1' file to then later use it in the 'run.ps1' file. So, this project isn't exactly reusable anywhere without modifications. Another limitation/improvement is that the IoC alerts and hits variables could be separated by log file to better understand where exactly the hits are. A final limitation/improvement is that the report could contain the most critical lines extracted from the log files, helping the user understand where to look first.

Ultimately this project is good for a small local system, but improvements would have to be made for a professional reusable tool in a production environment.